Raspberry Pi Pico 2 W Lesson: Build a Lo-Fi Guitar Effects Processor

Safe Audio Input, PWM Output, and First Effects Program

Chuck and Emily Neel

What Students Build

Students build a breadboard guitar effects processor around a Raspberry Pi Pico 2 W. The project takes an instrument-level guitar signal, shifts it into the safe voltage range for the Pico, samples it on an analog input, applies a simple digital effect, and sends audio back out through a filtered PWM output.

The first goal is not a studio-quality pedal. The first goal is a working signal path that teaches the full chain:

Stage	Job
Input jack	Receives the guitar signal from a normal 1/4-inch instrument cable.
Input circuit	Protects the Pico and shifts the guitar waveform around 1.65V.
Pico ADC	Reads the audio on `GP26` / `ADC0`.
Effect code	Changes the samples to make clean, fuzz, tremolo, bitcrush, or a 440 Hz test tone.
PWM output	Sends the processed signal out on `GP16`.
Output filter	Smooths the PWM signal and sends it to a guitar amp or powered speaker input.

Important Safety Rules

- Do not connect a guitar, amp, speaker output, headphone output, or 9V pedal supply directly to a Pico GPIO pin.
- A Pico input pin must stay between `GND` and `3V3`. Guitar audio normally swings above and below 0V, so it needs an input circuit first.
- The Pico cannot drive a speaker or headphones directly. The output must go to a guitar amp input, powered speaker input, audio interface input, or amplifier module.
- Never connect this project to a guitar amp's speaker output. Use the amp's normal instrument input only.
- Start with the amp volume low. Turn up slowly after the circuit is working.
- Have the circuit checked before plugging in a guitar or amplifier.

What You Need

Exact parts can vary. This lesson assumes a breadboard build with simple analog parts.

- 1 Raspberry Pi Pico 2 W, already flashed with MicroPython
- 1 breadboard and jumper wires
- 1 data-capable USB cable
- 1 regulated 5V breadboard power supply
- 1 mono 1/4-inch input jack for the guitar
- 1 mono 1/4-inch output jack for the amplifier input
- 1 LM358P dual op amp. The `P` version is the 8-pin DIP package that fits a breadboard.
- Resistors: $2 \times 100\text{k}\Omega$, $1 \times 1\text{M}\Omega$, $1 \times 10\text{k}\Omega$, $1 \times 1\text{k}\Omega$, $1 \times 100\text{k}\Omega$
- Capacitors: $1 \times 0.1\mu\text{F}$, $1 \times 1\mu\text{F}$, $1 \times 10\mu\text{F}$, $1 \times 10\text{nF}$

- Optional: LED plus 220Ω resistor for a status light on `GP15`
- Optional: button or footswitch connected to `GP14` for effect changes later
- Guitar, instrument cable, and a small amp or powered speaker with an input volume control

LM358P note: the LM358P is viable for this first lo-fi input buffer, but it is not rail-to-rail. For this lesson, power the LM358P from the regulated `+5V` breadboard rail, not from `3V3`. The guitar signal is still biased from the Pico's `3V3` rail so the Pico analog input rests near 1.65V.

Part note: if you have a ready-made audio codec, ADC/DAC, preamp, or amplifier board, you can use that instead of the simple input/output circuits below. The same software idea still applies, but the wiring will follow that board's pinout.

Why the Input Circuit Is Needed

A guitar pickup creates a small AC signal. AC means the voltage moves positive and negative around 0V. The Pico ADC cannot read negative voltage, so we create a quiet midpoint called `VBIAS`. For a 3.3V Pico, that midpoint is about 1.65V.

The input circuit does three jobs:

1. It AC-couples the guitar so DC from the circuit does not go back into the guitar.
2. It biases the guitar signal around 1.65V so the Pico ADC can read the whole waveform.
3. It buffers the guitar with an op amp so the pickup is not loaded down too heavily.

Power the Breadboard Rails

Use the same rail wiring you have been using:

1. Turn the 5V power supply off before wiring.
2. Connect the breadboard `+5V` rail to Pico physical pin 39, `VSYS`.
3. Connect the breadboard `GND` rail to Pico physical pin 38, `GND`.
4. Use Pico physical pin 36, `3V3(OUT)`, only for the bias circuit. Do not connect `3V3(OUT)` to the `+5V` rail.

Power check: before adding the guitar circuit, confirm that the `+5V` rail is near 5V and the Pico's `3V3(OUT)` pin is near 3.3V.

Build the 1.65V Bias Node

Turn off power before changing wiring.

1. Connect one 100kΩ resistor from Pico physical pin 36, `3V3(OUT)`, to a new breadboard row named `VBIAS`.
2. Connect another 100kΩ resistor from `VBIAS` to `GND`.
3. Connect the 10μF capacitor from `VBIAS` to `GND`. If it is polarized, the positive leg goes to `VBIAS`.
4. Connect the 0.1μF capacitor near the LM358P's power pins, from LM358P pin 8 to LM358P pin 4.

Before connecting the guitar, use a meter if one is available. `VBIAS` should be close to 1.65V.

Build the LM358P Guitar Input

Use channel A of the LM358P as a voltage follower. That means the op amp output connects back to its negative input, and the guitar signal goes into the positive input.

Connection	Where it goes
LM358P pin 8	Breadboard `+5V` rail, the same rail connected to Pico physical pin 39 / `VSYS`
LM358P pin 4	Breadboard `GND` rail
Input jack sleeve	`GND`
Input jack tip	One side of the 1μF input capacitor
Other side of input capacitor	LM358P pin 3, channel A positive input
1MΩ resistor	LM358P pin 3 to `VBIAS`
LM358P pin 1	LM358P pin 2, making channel A a voltage follower
LM358P pin 1	Through a 10kΩ protection resistor to Pico `GP26` / `ADC0`
Pico `AGND`	`GND` rail
Unused channel B	Tie LM358P pin 5 to `VBIAS`, and tie pin 6 to pin 7

Input capacitor note: if the 1μF input capacitor is polarized, put the positive leg toward the op amp positive input node.

Why not 3.3V for the LM358P? the LM358P can run from a low single supply, but its input and output do not reach all the way to the positive rail. With only `3V3` power, a 1.65V-biased audio signal has almost no headroom and may clip even before the Pico sees it. Powering the LM358P from the `+5V` rail gives this beginner circuit enough room for a small guitar signal.

Check before continuing: with no guitar plugged in, `GP26` should sit near 1.65V. If it is near 0V or 3.3V, stop and fix the bias circuit.

Build the PWM Audio Output

The Pico creates output audio by rapidly changing a PWM duty cycle. A resistor-capacitor filter smooths that fast PWM into an audio-like signal.

Connection	Where it goes
Pico `GP16`	Through a 1kΩ resistor to a row named `PWM_AUDIO`
10nF capacitor	`PWM_AUDIO` to `GND`
Output capacitor, 1μF	From `PWM_AUDIO` to the output jack tip
100kΩ resistor	Output jack tip to `GND`
Output jack sleeve	`GND`

If the 1μF output capacitor is polarized, put the positive leg toward `PWM_AUDIO`. The guitar amp side should be referenced to ground by the 100kΩ resistor.

Optional Status LED and Button

These are helpful for debugging.

- Status LED: `GP15` to the LED long leg, LED short leg through 220Ω to `GND`.
- Button or footswitch: one side to `GP14`, the other side to `GND`. The program can use `Pin.PULL_UP`.

Create the Program

In VS Code, create a new folder named `pico-guitar-effects`. Inside that folder, create a file named `main.py`.

Type this program. Keep the lines short and match the spelling carefully.

```
from machine import ADC, Pin, PWM
from time import ticks_add, ticks_diff, ticks_us

AUDIO_IN_PIN = 26
AUDIO_OUT_PIN = 16
LED_PIN = 15
```

```
SAMPLE_RATE = 8000
SAMPLE_PERIOD_US = 1000000 // SAMPLE_RATE
PWM_FREQUENCY = 125000
CENTER = 32768

# Try: "clean", "fuzz", "tremolo", "bitcrush", or "tone"
EFFECT = "fuzz"

FUZZ_GAIN = 3
FUZZ_LIMIT = 12000
BITCRUSH_SHIFT = 6
TREMOLO_HALF_PERIOD = SAMPLE_RATE // 8
TONE_FREQUENCY = 440
TONE_AMPLITUDE = 9000

audio_in = ADC(Pin(AUDIO_IN_PIN))
audio_out = PWM(Pin(AUDIO_OUT_PIN))
led = Pin(LED_PIN, Pin.OUT)

audio_out.freq(PWM_FREQUENCY)
audio_out.duty_u16(CENTER)


def clamp(value, low, high):
    if value < low:
        return low
    if value > high:
        return high
    return value


def process(sample, sample_number):
    if EFFECT == "tone":
        phase = (sample_number * TONE_FREQUENCY) % SAMPLE_RATE
        if phase < SAMPLE_RATE // 2:
            changed = TONE_AMPLITUDE
        else:
            changed = -TONE_AMPLITUDE
        return clamp(changed + CENTER, 0, 65535)

    centered = sample - CENTER

    if EFFECT == "clean":
        changed = centered
    elif EFFECT == "fuzz":
        changed = clamp(centered * FUZZ_GAIN,
                        -FUZZ_LIMIT, FUZZ_LIMIT)
    elif EFFECT == "tremolo":
        phase = sample_number % (TREMOLO_HALF_PERIOD * 2)
        if phase < TREMOLO_HALF_PERIOD:
            changed = centered
        else:
            changed = centered // 3
    elif EFFECT == "bitcrush":
        changed = (centered >> BITCRUSH_SHIFT) << BITCRUSH_SHIFT
    else:
        changed = centered
```

```
    return clamp(changed + CENTER, 0, 65535)


print("Pico guitar effects processor starting.")
print("Input: GP26 / ADC0. Output: GP16 PWM.")
print("Current effect:", EFFECT)

sample_number = 0
next_sample_at = ticks_us()
led.on()

try:
    while True:
        if EFFECT == "tone":
            raw = CENTER
        else:
            raw = audio_in.read_u16()

        output = process(raw, sample_number)
        audio_out.duty_u16(output)

        sample_number += 1
        next_sample_at = ticks_add(next_sample_at, SAMPLE_PERIOD_US)
        while ticks_diff(next_sample_at, ticks_us()) > 0:
            pass
finally:
    audio_out.duty_u16(CENTER)
    led.off()
```

Run the Program

1. Turn the amplifier volume all the way down.
2. Connect the Pico 2 W to the Mac with the USB cable.
3. In VS Code, open `main.py`.
4. Use `MicroPico: Run current file on Pico`.
5. Plug the guitar into the input jack.
6. Plug the output jack into a small amp or powered speaker input.
7. Slowly raise the amp volume while playing the guitar.

If the circuit and code are working, you should hear a rough fuzz effect. It may sound noisy or lo-fi. That is expected for this first breadboard version.

Change the Sound

Stop the program before editing it. Change the `EFFECT` line near the top:

```
EFFECT = "clean"
EFFECT = "fuzz"
EFFECT = "tremolo"
EFFECT = "bitcrush"
EFFECT = "tone"
```

Run the file again after each change.

- `clean`: passes the signal through with very little processing.

- `fuzz`: boosts the signal and clips it for a distorted sound.
- `tremolo`: rapidly switches between loud and quiet.
- `bitcrush`: removes detail from the waveform for a digital sound.
- `tone`: generates a 440 Hz test tone and ignores the guitar input.

What the Program Means

- `ADC(Pin(26))` reads the analog input on `GP26`.
- `PWM(Pin(16))` creates a fast output pulse train on `GP16`.
- `CENTER = 32768` is the middle of the 16-bit audio range.
- `sample - CENTER` changes the ADC reading into a signed audio sample.
- The `process()` function is where the effect happens.
- `tone` mode is useful for testing the output filter, output buffer, output jack, and amp connection before the guitar input works.
- `duty_u16(output)` sends the processed sample to the PWM output.
- The timing loop tries to run at about 8000 samples per second.

Debugging Checkpoints

1. With power on and no guitar plugged in, `VBIAS` should be about 1.65V.
2. `GP26` should also sit near 1.65V when there is no input signal.
3. The status LED should turn on when the program starts.
4. The full `pico-guitar/main.py` project prints ADC input stats in the MicroPico console about once per second while you are not in `tone` mode.
5. Touching the input jack tip may create a buzz in the amp. Keep volume low.
6. If clean mode works but fuzz is too harsh, lower `FUZZ_GAIN` or raise `FUZZ_LIMIT`.
7. If the audio is very quiet, check that the output capacitor and output jack ground are wired correctly.
8. If the sound is squealing or unstable, move wires apart and keep input wires away from output wires.

Console input check: when you play the guitar, `peak` and `span` should grow. `status=active` means the Pico is seeing a reasonable signal. `status=quiet` or `status=weak` means the signal is barely moving. `status=CENTER_OFF` usually means the 1.65V bias is wrong. `status=CLIPPING_OR_RAIL` usually means the signal is hitting 0V or 3.3V.

Troubleshooting

- **No sound at all:** check amp volume, output jack sleeve to `GND`, and `GP16` wiring.
- **Very loud hum:** check that the input jack sleeve, output jack sleeve, Pico `GND`, and op amp `GND` are connected together.
- **The guitar sounds thin:** check that the 1MΩ input resistor goes to `VBIAS`, not directly to `GND`.
- **The Pico disconnects or resets:** unplug immediately and check for shorts between `3V3` and `GND`.
- **The sound is harsh in every mode:** the input may be clipping before the Pico. Lower the guitar volume or reduce op amp gain if you added gain.
- **The code runs but audio is choppy:** lower `SAMPLE_RATE` to 6000 or 4000. MicroPython timing is limited.
- **A line says `ImportError`:** make sure the file is running on the Pico, not on the Mac.

Extensions

After the basic version works:

- Add a potentiometer to `GP27` and use it to control fuzz gain.
- Use the button on `GP14` to switch effects without editing code.
- Replace the PWM output with an external DAC or I2S audio codec.
- Rewrite the audio loop in C or use PIO/DMA for lower latency and cleaner sound.
- Put the circuit in a metal enclosure with proper jacks and strain relief.

Key Takeaway

The Pico 2 W is a digital board, but a guitar is an analog instrument. A useful effects processor needs both sides: an analog input/output circuit that protects the hardware, and software that changes the sound in real time.